

TECHNICAL WHITEPAPER & SYSTEM REPORT

AeroBook Flight Platform

Complete Architectural Specification & Technology Analysis

A comprehensive engineering deep dive into cloud-native microservices vs monolithic trade-offs, technology stack selection rationale, domain service encapsulation, security governance, and future evolutionary scope.

SYSTEM SPECIFICATIONS & PARAMETERS

Architecture Pattern: Cloud Microservices (Spring)	Database Pattern: Database-per-Service (MySQL 8)	Messaging Engine: RabbitMQ (AMQP 0-9-1)
Service Discovery: Netflix Eureka (AP Model)	Edge Routing: Spring Cloud Gateway (Netty)	Security Standard: Stateless JWT (HMAC-SHA256)

EXECUTIVE ARCHITECTURAL SUMMARY

AeroBook represents a modern cloud-native response to the fragility, scalability bottlenecks, and single-point-of-failure risks inherent in legacy monolithic airline reservation systems. Engineered using Java 17 and Spring Boot 3.2, the platform decomposes airline flight discovery, dynamic surge pricing, atomic seat reservations, and airport boarding into autonomous microservices.

By strictly implementing the Database-per-Service pattern, AeroBook ensures that high-volume flight search reads never cause database lock contention on transactional booking manifests. Non-blocking edge routing via Netty WebFlux, asynchronous event-driven notifications via RabbitMQ, and client-side payment tokenization (zero credit card storage) establish an enterprise-grade operational benchmark for high-availability aviation software.

TABLE OF CONTENTS

Section 2	Why Microservices & Not Monolithic?	- Monolithic failure modes, blast radius containment, scalability comparison.
Section 3	Technology Stack Selection Rationale	- Why Spring Boot (not Node/Django/Go), Gateway (not Zuul/Nginx), MySQL & RabbitMQ.
Section 4	Domain Services Deep Dive (Part 1)	- Service-by-service breakdown: Gateway, Eureka, Auth, and User services.
Section 5	Domain Services Deep Dive (Part 2)	- Service-by-service breakdown: Flight, Fare, Booking Core, and Check-In services.
Section 6	Data Architecture, Security & Governance	- Database isolation, PCI-DSS zero-PAN storage, two-phase seat locking, and Saga pattern.
Section 7	Future Evolutionary Roadmap	- AI dynamic pricing models, biometric gate boarding, and IATA ONE Order interlining.
Section 8	Viva, Defense & Technical Review FAQ	- Examiner questions, architecture defense, and key engineering trade-offs.

Why Microservices & Why Not Monolithic?

Architectural Trade-Offs, Failure Containment & Domain Decomposition

In commercial airline operations, system downtime carries devastating financial penalties and regulatory sanctions. Traditional monolithic architectures consolidate all business functions—authentication, flight search, pricing, bookings, and check-in—into a single unified codebase sharing a centralized relational database. While simpler to deploy initially, this model introduces catastrophic systemic vulnerabilities.

THE MONOLITHIC FAILURE CASCADE

1. Single Point of Failure (SPOF):
A memory leak or unhandled NullPointerException in dynamic fare calculation crashes the entire JVM. Passengers at airport boarding gates cannot board because the check-in servlet shares the same crashed memory space.
2. Shared Database Contention:
During holiday promotional sales, 100,000 users query flight routes. The shared database connection pool exhausts, locking customer tables and causing payment transactions to time out.
3. High Deployment Blast Radius:
Updating an airport departure tax rule requires rebuilding, testing, and deploying the entire monolith, risking regressions across booking engines.

THE AEROBOOK MICROSERVICES SOLUTION

1. Absolute Fault Isolation:
Microservices operate in isolated container runtimes. If fare-service crashes, booking-service continues using cached published base fares. Airport gate scanners operate unimpeded on their dedicated check-in database.
2. Targeted Asymmetric Scaling:
Flight catalog search experiences a 200:1 read-to-write ratio over bookings. In AeroBook, we scale flight-service to 10 instances while keeping booking-service at 2 instances, optimizing cloud hosting costs.
3. Zero Database Contention:
Enforces Database-per-Service. Dedicated MySQL schemas ensure flight search reads never lock booking checkout tables.

ARCHITECTURAL COMPARISON MATRIX

Evaluation Dimension	Monolithic Architecture	AeroBook Microservices
System Blast Radius	100% (Catastrophic failure)	Isolated (<15% per service container)
Database Contention	High (Shared tables & connections)	Zero (Isolated MySQL schema per service)
Horizontal Scalability	Coarse (Must duplicate whole app)	Fine-grained (Scale flight search 10x)
Deployment Agility	Slow (Multi-hour full regression)	Rapid (Continuous service-level deployments)
Network Overhead	Negligible (In-memory JVM calls)	Moderate (HTTP REST / AMQP serialized hops)
Data Consistency	Immediate ACID (Single database)	Eventual Consistency (Saga + RabbitMQ AMQP)
Operational Complexity	Low (Single JAR/WAR artifact)	Moderate (Eureka discovery, Gateway routing)

Engineering Verdict: While microservices introduce distributed network hops and operational complexity, AeroBook mitigates this through non-blocking Netty gateway routing, automated Eureka discovery, and asynchronous RabbitMQ event streaming—delivering the fault isolation and scalability required for commercial airline operations.

Technology Stack: "Why This & Not Others" Analysis

Defending the Technology Decisions Behind AeroBook Engine

BACKEND FRAMEWORK

Spring Boot 3.2 & Java 17

- Why Chosen:** Spring Data JPA/Hibernate 6 provides declarative transaction management (@Transactional), automated schema synchronization, and robust enterprise actuators. Java 17 LTS introduces records for immutable DTOs, enhanced garbage collection (G1/ZGC), and strong compile-time type safety for financial booking calculations.
- Alternatives:** Node.js / Express, Python / Django, Go (Golang) — Node.js single-threaded event loop bottlenecks under heavy mathematical seat matrix locking; lacks declarative multi-table transactions. Python GIL limits high-concurrency throughput and tail latencies. Go lacks enterprise ORM maturity and requires massive networking/validation boilerplate.

EDGE ROUTING & SECURITY

Spring Cloud Gateway (Netty WebFlux)

- Why Chosen:** Non-blocking reactive architecture built on Project Reactor. Handles tens of thousands of concurrent client connections over Netty event loops without allocating 1 OS thread per request. Integrates seamlessly with Spring Security filters and Eureka discovery.
- Alternatives:** Netflix Zuul 1.x, NGINX / Kong — Zuul 1.x relies on a blocking servlet model where a single slow downstream service exhausts all gateway worker threads. NGINX/Kong require writing custom Lua plugins or proprietary enterprise licenses to integrate with Eureka heartbeats and Spring JWT contexts.

SERVICE DISCOVERY

Netflix Eureka Registry (Port 8761)

- Why Chosen:** High-availability AP system in CAP theorem. Microservices dynamically register on startup and emit heartbeats every 30s. Self-preservation mode prevents premature unregistration during transient network glitches. Integrates natively with Spring Cloud LoadBalancer.
- Alternatives:** HashiCorp Consul, Apache ZooKeeper, K8s DNS — Consul and ZooKeeper are CP systems that halt registration during network partitions to elect leaders, causing catastrophic routing blackouts. Pure Kubernetes DNS requires running a full Kubernetes cluster and lacks lightweight local Docker Compose portability.

RELATIONAL DATABASE

MySQL 8.0 (Database-per-Service)

- Why Chosen:** Strict ACID transaction guarantees essential for airline financial records and seat bookings. Pessimistic row-level locking (SELECT ... FOR UPDATE) enables atomic 10-minute temporary seat holds. InnoDB engine provides MVCC concurrency control.
- Alternatives:** MongoDB / Document NoSQL, PostgreSQL — MongoDB lacks native multi-document relational constraints; eventual consistency models risk overselling seats during flash fare sales. MySQL 8.0 provides a smaller memory footprint per container when running 6 separate database containers simultaneously.

ASYNCHRONOUS MESSAGING

RabbitMQ (AMQP 0-9-1 Broker)

- Why Chosen:** Push-based message broker optimized for transactional task queues. Direct exchange aerobook.direct bound to queue booking.events offloads itinerary email generation (SendGrid) and SMS alerts (Twilio) in <5ms without blocking user checkout.
- Alternatives:** Apache Kafka — Kafka is an event streaming log designed for massive big-data ingestion and stream replay via consumer pull offsets. RabbitMQ is vastly superior for transactional worker queues requiring granular message acknowledgments, dead-letter exchanges (DLX), and zero cluster setup complexity.

Domain Services Deep Dive (Part 1)

Core Infrastructure, Identity Management & Customer Profiles

DISCOVERY ENGINE

eureka-server (Port 8761)

Centralized Microservices Registry & Health Monitor

Acts as the telephone directory for the distributed platform. When any microservice boots up, it registers its dynamic port and IP address. Services emit heartbeat pings every 30 seconds to refresh lease availability.

- Key Endpoints:**
- GET /eureka/apps (Fetch registered instances) | Actuator Health Probes (/actuator/health)
- Database Store:**
- None (In-Memory ConcurrentHashMap with Peer-to-Peer Replication)
- Why Isolated:**
- Eliminates hardcoded IP addresses. Enables elastic auto-scaling and zero-downtime rolling updates.

EDGE ROUTER

api-gateway (Port 8083)

Non-Blocking Netty Reverse Proxy & Security Checkpoint

Single entry point for all web and mobile clients. RouteValidator whitelists public routes (/api/auth/**, /api/flights/search). Secured routes pass through AuthenticationFilter, validating Bearer JWT tokens in memory before forwarding requests.

- Key Endpoints:**
- GET /v3/api-docs (Dynamic OpenAPI Hub) | GatewayDocsController dropdown aggregator
- Database Store:**
- None (Stateless Gateway)
- Why Isolated:**
- Shields internal microservices from direct public Internet exposure, centralizing SSL termination, CORS, and rate limiting.

IDENTITY PROVIDER

auth-service (Port 8082)

User Authentication, BCrypt Hashing & JWT Issuance

Handles user registration and authentication. Plaintext passwords are salted and hashed using Spring Security BCryptPasswordEncoder (strength 10). Upon successful login, issues a signed stateless HMAC-SHA256 JWT containing user roles and a 24-hour expiration timestamp.

- Key Endpoints:**
- POST /api/auth/register | POST /api/auth/login | POST /api/auth/validate
- Database Store:**
- aerobook_auth_db (users, roles, user_roles, refresh_tokens)
- Why Isolated:**
- Keeps sensitive user credentials and password hashes completely isolated from business booking services.

CUSTOMER PROFILE

user-service (Port 8084)

Traveler Records, Passports & Preferences Store

Maintains comprehensive passenger identity records, international passport details, date of birth, emergency contacts, and frequent flyer mileage accounts. Enforces international passport format validation and GDPR profile data rectification.

- Key Endpoints:**
- GET /api/users/profile | PUT /api/users/profile | GET /api/users/{id}/passports
- Database Store:**
- aerobook_user_db (user_profiles, passports, addresses)
- Why Isolated:**
- Encapsulates Personally Identifiable Information (PII) to comply with international data privacy regulations.

Domain Services Deep Dive (Part 2)

Operational Core: Flight Discovery, Dynamic Pricing, Booking Saga & Boarding

FLIGHT & SEAT INVENTORY

flight-service (Port 8087)

Fleet Scheduling, Seat Maps & Atomic Seat Locking

Executes high-speed flight queries indexed on (origin, destination, departure). Manages aircraft seat configurations. Implements two-phase atomic seat hold locking with 10-minute hold expirations to prevent double-booking during checkout.

Key Endpoints: GET /api/flights/search | GET /api/flights/{id}/seats | POST /api/flights/{id}/seats/lock

Database Store: aerobook_flight_db (flights, aircraft, seats, seat_inventories)

Why Isolated: Handles 200x more read traffic than booking checkout. Isolating it prevents flight search queries from locking booking databases.

DYNAMIC PRICING ENGINE

fare-service (Port 8089)

Real-Time Surge Pricing & Multiplier Calculations

Computes real-time dynamic fares using formula: $\text{FinalPrice} = \text{BaseFare} * \text{CabinMultiplier} * (1 + \text{DemandSurge}) + \text{Taxes}$. Calculates surge from cabin occupancy (0-50% = 0%, 51-80% = +15%, 81-100% = +35%). Uses BigDecimal to prevent floating-point financial drift.

Key Endpoints: POST /api/fares/calculate | GET /api/fares/{flightid}/quote | PUT /api/fares/rules

Database Store: aerobook_fare_db (fares, surge_multipliers, class_pricing_rules)

Why Isolated: Enables revenue managers to adjust dynamic surge rules and fuel surcharges instantly without redeploying booking services.

CORE TRANSACTION SAGA

booking-service (Port 8088)

Transactional PNR Orchestrator & Saga Coordinator

Coordinates multi-passenger bookings. Validates seat holds via flight-service, confirms fare quotes via fare-service, authorizes payments with Payment Gateway, generates unique 6-character PNRs, and emits BookingConfirmedEvent to RabbitMQ.

Key Endpoints: POST /api/bookings | GET /api/bookings/{pnr} | POST /api/bookings/{pnr}/cancel

Database Store: aerobook_booking_db (bookings, passengers, booking_payments, audit_logs)

Why Isolated: Protects the core financial revenue pipeline. Demands strict ACID isolation and independent database resources.

AIRPORT GATE ENGINE

check-in-service (Port 8090)

Airport Web Check-In & 2D QR Boarding Pass Scanner

Enforces flight departure window rules (T-24h to T-1h). Validates PNR status with booking-service. Generates digital boarding passes with encrypted 2D QR codes containing HMAC-SHA256 signatures. Gate scanner API validates QR codes and updates status to EMBARKED.

Key Endpoints: POST /api/checkin/initiate | GET /api/checkin/{pnr}/pass | POST /api/checkin/gate-scan

Database Store: aerobook_checkin_db (check_ins, boarding_passes, gate_verifications)

Why Isolated: Airport gate scanners require sub-100ms response times and 99.999% uptime, even during online booking website maintenance.

Data Architecture, Security & Governance

PCI-DSS Compliance, Concurrency Control & The Saga Pattern

1. ZERO PAN STORAGE (PCI-DSS COMPLIANCE MINIMIZATION)

Under PCI-DSS compliance, any database storing 16-digit credit card Primary Account Numbers (PAN) requires expensive hardware security modules (HSM) and rigorous quarterly forensic audits. AeroBook achieves complete audit scope minimization:

- Front-end web/mobile clients communicate directly with third-party payment gateways (Stripe/Razorpay) via client SDKs.
- The gateway tokenizes card details and returns an opaque, single-use payment token (tok_visa_4242).
- booking-service submits this token for settlement and only receives a non-sensitive settlement reference (txn_id).
- Zero credit card numbers or CVVs ever touch AeroBook memory, application logs, or MySQL database disks.

2. ATOMIC SEAT LOCKING & DOUBLE-BOOKING PREVENTION

During promotional fare sales, concurrent requests for the same seat cause race conditions unless managed by pessimistic row-level locking:

- When a user selects Seat 14B, flight-service executes: `SELECT * FROM seat_inventories WHERE seat_id = :id FOR UPDATE`.
- The first transaction acquires an exclusive row lock, verifies status = AVAILABLE, transitions status to HELD, and sets a 10-minute hold expiration timestamp (`hold_expires_at = NOW() + 10 MINUTES`).
- Subsequent concurrent transactions are blocked until the lock releases, reading status = HELD and returning HTTP 409 Conflict.
- If checkout times out or payment fails, a scheduled worker automatically resets the seat status to AVAILABLE.

3. DISTRIBUTED TRANSACTIONS & THE SAGA PATTERN

In microservices, traditional two-phase commit (2PC) distributed database transactions introduce severe blocking latencies and single points of failure. AeroBook orchestrates distributed consistency using the Saga Pattern:

- Step 1: booking-service requests an atomic seat lock from flight-service via OpenFeign REST.
- Step 2: booking-service calls Payment Gateway to settle the authorized charge (\$).
- Step 3: If payment succeeds, booking-service commits the PNR record to `aerobook_booking_db` and emits `BookingConfirmedEvent` to RabbitMQ.
- Step 4: If payment fails, a compensating transaction is immediately triggered to release the seat hold in flight-service.

4. STATELESS JWT TOKEN SECURITY & VALIDATION

- Clients authenticate once via auth-service, receiving a cryptographically signed HMAC-SHA256 JWT bearer token.
- The token payload encapsulates user identity, authority claims ([`ROLE_PASSENGER`, `ROLE_ADMIN`]), and expiration timestamps.
- api-gateway validates token integrity in memory using shared secret keys without querying databases for every inbound request.
- Downstream microservices receive pre-authenticated claims via HTTP headers (`X-User-Id`, `X-User-Roles`), ensuring zero database auth overhead.

Future Scope & Evolutionary Roadmap

Next-Generation Enhancements for Modern Aviation Operations

01

AI/ML Dynamic Pricing Engine (Revenue Management)

Deploy a specialized Python/FastAPI microservice running an XGBoost regression model to forecast passenger booking demand curves. The model will analyze historical booking velocities, local holiday schedules, competitor web scraper feeds, and macroeconomic fuel prices to dynamically adjust seat prices per minute, maximizing revenue per available seat kilometer (RASK).

02

Biometric Airport Gate Embarkation (Facial Recognition)

Integrate airport departure gate cameras with high-speed facial recognition edge inference engines. Passenger facial biometric embeddings captured at boarding gates will be matched against encrypted passport photo vectors stored in user-service, enabling frictionless, hands-free gate boarding without requiring physical paper or phone screens.

03

IATA ONE Order & NDC Multi-Carrier Interlining

Upgrade from legacy PNR/e-ticket standards to the modern IATA ONE Order XML/JSON specification. This enables seamless multi-carrier flight bookings across partner airline alliances with a single unified booking reference, unified baggage RFID telemetry tracking, and automated inter-airline revenue clearing.

04

Mobile Wallet Pass Integration (Apple Wallet & Google Wallet)

Direct integration with Apple Wallet (PKPass) and Google Wallet APIs. Boarding passes generated by check-in-service will automatically install into passenger mobile wallets, featuring live background push notifications for real-time departure gate changes, boarding group calls, and weather delay alerts.

05

Blockchain Distributed Ledger for Baggage Custody Transfer

Implement a consortium distributed ledger (Hyperledger Fabric) tracking baggage custody transfers between ground handlers, airport apron operations, connecting flights, and baggage carousels, virtually eliminating lost luggage incidents.

Technical Defense, Viva Review & Examiner FAQ

Essential Answers for Evaluators, Examiners & Architecture Reviews

Q1: Why did you choose Database-per-Service instead of a single shared database?

A shared database creates a single point of failure and tightly couples services at the schema level. If multiple services share a database, high-volume flight search queries create connection pool contention and row locks that block booking commits. Furthermore, Database-per-Service allows each microservice database to scale, backup, and migrate schemas independently.

Q2: How does the system prevent double-booking when multiple users attempt to book the same seat?

flight-service implements pessimistic row-level locking (SELECT ... FOR UPDATE) inside an atomic database transaction. The first user to select the seat locks the row, changes status to HELD, and sets a 10-minute expiration timestamp. Subsequent concurrent requests are blocked until the lock commits, reading status = HELD, and are rejected with HTTP 409 Conflict.

Q3: Why is RabbitMQ used instead of synchronous HTTP REST calls for sending notifications?

Third-party email (SendGrid) and SMS (Twilio) APIs typically have latencies of 500ms to 3,000ms and can experience temporary rate limits. Calling them synchronously within the booking request would slow checkout response times and risk transaction failure if SendGrid is slow. Publishing an event to RabbitMQ takes <5ms, allowing background workers to handle dispatch with automatic retries.

Q4: How do you handle distributed data consistency without cross-database foreign keys?

AeroBook uses Eventual Consistency guided by the Saga Pattern. Instead of distributed two-phase commits (which cause severe performance degradation), services reference immutable IDs (e.g. flightId, pnr) validated via OpenFeign REST calls. If payment or downstream processing fails, compensating transactions release the held seats immediately.

Q5: Why is Spring Cloud Gateway built on Netty WebFlux instead of traditional Tomcat Servlets?

Tomcat uses a thread-per-request model that exhausts OS thread pools at ~2,000 concurrent connections. Netty WebFlux uses non-blocking event loops where a small fixed number of threads handle tens of thousands of concurrent connections asynchronously, providing high resilience under flash-sale traffic surges.

PROJECT EVALUATION CONCLUSION

AeroBook successfully demonstrates a modern, enterprise-grade distributed aviation platform. Through strict database isolation, non-blocking edge routing, PCI-DSS compliance, and asynchronous AMQP messaging, it resolves the systemic vulnerabilities of legacy monolithic reservation systems.

